

# Reviewer Pack — Minimal Root Separation of Matrix pencils and BP\_ReadTwice L...

Entry a56d096468e6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-21 15:35:02 UTC

## Minimal Root Separation of Matrix pencils and BP\_ReadTwice Lower Bounds

**Entry ID:** a56d096468e6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-21 15:35:02 UTC

### 1. Conjecture

**Field A** (mathematical branch): Real Algebraic Geometry **Field B** (complexity object): Branching program: read-once vs read-twice (BP)

#### Statement:

[‘For any read-twice BP  $P$ , the minimal root separation of its characteristic matrix pencil is bounded by the logarithm of the size of  $P$ :  $\log(\text{size}(P)) \leq \text{min\_root\_separation}(P)$ .’, ‘Furthermore, for the trivial  $\text{IP}_2$  BP, this lower bound is at least  $\Omega(n)$ .’, ‘For any CNF tautology representing the language computed by a read-twice BP, its minimal root separation satisfies the above bounds.’]

#### Rationale (proposer’s reasoning):

[‘Real algebraic geometry provides a framework for studying the properties of matrix pencils. By analyzing the roots of the characteristic polynomial of a matrix pencil associated with a BP, we can derive lower bounds on the complexity of the BP.’, ‘The minimal root separation is an invariant that captures the geometric structure of the roots and can be computed efficiently. This makes it a viable candidate for deriving complexity-theoretic lower bounds.’, ‘Since read-twice BPs are known to have exponential size lower bounds for  $\text{IP}_2$ , using this real algebraic geometry tool should allow us to distinguish them from unrestricted BPs.’]

**Taxonomy category:** META\_COMPLEXITY (status at proposal time: alive)

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 9f99a1fbd063685a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if, for all BP instances up to size 40, the computed minimal root separation meets  $\log(\text{size}(P)) \leq \text{min\_root\_separation}(P)$  with at least 80% of the seeds and a mean value  $\leq 3$ . It is falsified if any seed or any instance has a minimal root separation exceeding  $\log(\text{size}(P)) + 3$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict   | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|-----------|------------|------------------|------------------|
| RELATIVIZATION | UNCERTAIN | 1.00       | HITS             | SAFE             |
| NATURAL_PROOFS | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE      | 1.00       | UNCERTAIN        | SAFE             |
| KARP_LIPTON    | SAFE      | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "Minimal Root Separation" AND "Matrix pencil" AND BP\_ReadTwice - "Real Algebraic Geometry" AND "read-once vs read-twice BP" AND "root separation" - "logarithm of size P" AND "BP" AND "real algebraic geometry"

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            # Find pivot row
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]

            # Eliminate non-pivot elements
            for j in range(n):
                if i != j:
                    factor = A[j][i] / A[i][i]
                    for k in range(n + 1):
                        A[j][k] -= factor * A[i][k]

    def characteristic_polynomial(A):
        n = len(A)
        x = Fraction('x')
        det = Fraction(1)
        for i in range(n):
            det *= (A[i][i] - x)
```

```

    return det

def min_root_separation(poly):
    roots = []
    for coeff, exp in poly.items():
        if coeff != 0:
            root = Fraction(coeff) ** (-Fraction(1, exp))
            roots.append(root)
    return min(abs(r1 - r2) for r1, r2 in itertools.combinations(roots, 2))

def generate_random_cnf(size):
    cnf = []
    for _ in range(size):
        clause = [random.randint(-size, size) for _ in range(random.randint(1, size))]
        cnf.append(clause)
    return cnf

n = random.choice([5, 10, 15, 20, 30, 40])
cnf = generate_random_cnf(n)

A = [[0] * (n + 1) for _ in range(n)]
B = [[0] * (n + 1) for _ in range(n)]

for clause in cnf:
    for literal in clause:
        if literal > 0:
            row, col = divmod(literal - 1, n)
            A[row][col] += 1
            B[row][col] -= 1
        else:
            row, col = divmod(-literal - 1, n)
            A[row][n] += 1
            B[row][col] += 1

gaussian_elimination(A)
char_poly = characteristic_polynomial(A)

min_separation = min_root_separation(char_poly)
log_size = math.log(n)

return {
    "metric_name": "min_root_separation",
    "metric_value": min_separation,
    "instances_tested": 1,
    "conjecture_holds": min_separation <= log_size + 3,
    "counterexample": "" if min_separation <= log_size + 3 else f"n={n}, min_separation={min_s
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\n={results[0]['instances_tested']}, min_separat

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b66c72a.py", line 98, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b66c72a.py", line 78, in run_trial
    gaussian_elimination(A)
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7b66c72a.py", line 34, in gaussian_elim
    factor = A[j][i] / A[i][i]
              ~~~~~^~~~~~
ZeroDivisionError: float division by zero

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Re-run the test without errors to verify the conjecture's validity.

## 11. Audit log (LLM calls)

**Total LLM calls:** 9

| # | Phase           | Provider      | Model       | Tokens in | out | Latency (ms) |
|---|-----------------|---------------|-------------|-----------|-----|--------------|
| 1 | propose         | ollama_remote | glm4:latest | 0         | 0   | 16632        |
| 2 | preregistration | ollama_remote | glm4:latest | 0         | 0   | 9307         |

| # | Phase    | Provider      | Model            | Tokens in | out | Latency (ms) |
|---|----------|---------------|------------------|-----------|-----|--------------|
| 3 | novelty  | ollama_remote | glm4:latest      | 0         | 0   | 8630         |
| 4 | novelty  | ollama_remote | glm4:latest      | 0         | 0   | 8990         |
| 5 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17066        |
| 6 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 7920         |
| 7 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 26280        |
| 8 | test_gen | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 13117        |
| 9 | judge    | ollama_remote | glm4:latest      | 0         | 0   | 54496        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162438 ms total latency. Provider mix: {'ollama\_remote': 9}

(full prompt+response transcripts available in *research/audit/a56d096468e6.jsonl*)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/a56d096468e6.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** *research/pvsnp\_sandbox/test\_\*.py* (per-cycle)

**Replay tarball:** *research/replay/a56d096468e6.tar.gz* (if generated)